

---

# Lab 3: Sensors and Reliable Measurement

## Photoresistor Calibration and Ultrasonic Distance

ENGR 1105 | Fall 2026

---

---

| Name | Partner |
|------|---------|
| Date | Section |

---

### Learning Objectives

---

By the end of this lab, you should be able to:

1. build and test a photoresistor voltage-divider circuit;
2. collect light-sensor readings under known reference conditions;
3. choose a threshold from measured data and explain hysteresis;
4. wire an HC-SR04 ultrasonic distance sensor;
5. calculate distance from echo travel time;
6. identify and reject invalid distance readings;
7. average repeated measurements; and
8. create a multi-zone distance warning system.

## 1. Lab Organization

Lab 3 is completed in two class meetings:

---

| Part    | Main work                                                                                             |
|---------|-------------------------------------------------------------------------------------------------------|
| Lab 3.1 | Photoresistor circuit, light measurements, calibration, threshold control, and hysteresis             |
| Lab 3.2 | Ultrasonic wiring, distance calculation, validation, averaging, warning zones, and a design challenge |

---

## 2. Materials

---

| Item                                     | Purpose                                  |
|------------------------------------------|------------------------------------------|
| Arduino Uno and USB cable                | Controller, power, and program upload    |
| Solderless breadboard and jumper wires   | Temporary circuit construction           |
| Photoresistor and 10 k $\Omega$ resistor | Light-sensing voltage divider            |
| HC-SR04 ultrasonic sensor                | Noncontact distance measurement          |
| LED and 330 $\Omega$ resistor            | Visual output                            |
| Computer with Arduino IDE                | Code editing, upload, and Serial Monitor |
| Ruler or tape measure                    | Distance reference                       |
| Flashlight or phone light                | Light-level reference                    |

---

### Before Applying Power

---

1. Disconnect USB power before making major wiring changes.
2. Never connect 5V directly to GND.
3. Use the 10 k $\Omega$  resistor in the photoresistor divider.
4. Use the 330  $\Omega$  resistor in series with the LED.
5. Check the HC-SR04 pin labels before connecting power.
6. Remove power immediately if the Arduino or a component becomes hot.

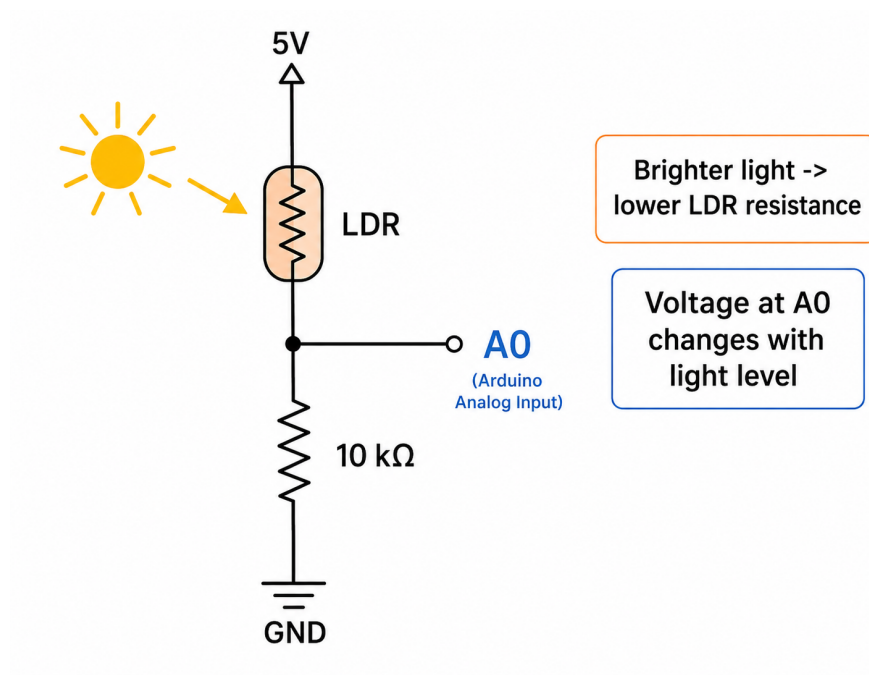
# Lab 3.1

## Photoresistor Calibration and Thresholds

Measuring Light and Creating Stable Decisions

### 3. Part A: Build the Photoresistor Circuit

A photoresistor, also called an LDR, changes resistance with light. In the circuit below, brighter light normally lowers the LDR resistance and raises the voltage measured at A0.



### Wiring Steps

1. Disconnect the Arduino from USB power.
2. Connect one photoresistor lead to Arduino 5V.
3. Connect the other photoresistor lead to a shared junction row on the breadboard.
4. Connect the 10 kΩ resistor from the junction row to GND.
5. Connect the junction row to analog pin A0.
6. Inspect the wiring before reconnecting USB power.

### Component Orientation

A photoresistor is not polarized, so either lead may face 5V. The circuit direction depends on whether the LDR is above or below the fixed resistor. Use the arrangement shown in this handout.

## 4. Part B: Read the Light Sensor

```
1 const int lightPin = A0;
2
3 void setup() {
4   Serial.begin(9600);
5 }
6
7 void loop() {
8   int lightValue = analogRead(lightPin);
9   Serial.println(lightValue);
10  delay(100);
11 }
```

### Instructions

---

1. Upload the program.
2. Open the Serial Monitor and set the baud rate to 9600.
3. Cover the sensor completely and wait for the reading to stabilize.
4. Record five covered readings.
5. Expose the sensor to normal room light and record five readings.
6. Illuminate the sensor with a flashlight and record five readings.

### Light Measurements

---

| Condition  | 1 | 2 | 3 | 4 | 5 | Average |
|------------|---|---|---|---|---|---------|
| Covered    |   |   |   |   |   |         |
| Room light |   |   |   |   |   |         |
| Flashlight |   |   |   |   |   |         |

---

Does brighter light produce a larger or smaller Arduino reading?

---

What is the approximate useful range between your covered and flashlight averages?

---

### Checkpoint 1: Light Sensor

---

Demonstrate that the reading changes clearly among covered, room-light, and flashlight conditions. Explain which circuit node is connected to A0.

## 5. Part C: Choose a Threshold

A threshold separates two operating conditions. Choose a threshold approximately halfway between the covered and room-light averages:

$$\text{threshold} = \frac{\text{covered average} + \text{room-light average}}{2}.$$

### Threshold Selection

---

Covered average: \_\_\_\_\_

Room-light average: \_\_\_\_\_

selected threshold = \_\_\_\_\_

Show the calculation:

---

---

---

Use your measured threshold in the program:

```
1  const int lightPin = A0;
2  const int ledPin = 9;
3  const int lightThreshold = 350;  // replace
4
5  void setup() {
6    pinMode(ledPin, OUTPUT);
7    Serial.begin(9600);
8  }
9
10 void loop() {
11   int lightValue = analogRead(lightPin);
12   Serial.println(lightValue);
13
14   if (lightValue < lightThreshold) {
15     digitalWrite(ledPin, HIGH);
16   } else {
17     digitalWrite(ledPin, LOW);
```

```
18 | }  
19 | }
```

### Test the Decision

1. Replace 350 with your measured threshold.
2. Upload the program.
3. Cover and uncover the photoresistor.
4. Confirm that the LED changes state.
5. If the behavior is reversed, reverse the comparison sign.

### Threshold Results

| Condition          | Sensor value | LED state |
|--------------------|--------------|-----------|
| Covered            |              |           |
| Near the threshold |              |           |
| Bright light       |              |           |

Why might the LED flicker when the reading is close to the threshold?

---

---

---

### Checkpoint 2: Calibrated Threshold

Demonstrate light-controlled LED behavior. Explain how your measured data was used to select the threshold.

## 6. Part D: Add Hysteresis

Use two thresholds so the LED does not repeatedly switch near one boundary.

```
1  const int lightPin = A0;
2  const int ledPin = 9;
3
4  const int turnOnThreshold = 330;    // replace
5  const int turnOffThreshold = 390;  // replace
6  bool ledOn = false;
7
8  void setup() {
9      pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13     int lightValue = analogRead(lightPin);
14
15     if (!ledOn && lightValue < turnOnThreshold) {
16         ledOn = true;
17     }
18
19     if (ledOn && lightValue > turnOffThreshold) {
20         ledOn = false;
21     }
22
23     digitalWrite(ledPin, ledOn ? HIGH : LOW);
24 }
```

Choose two thresholds separated by approximately 30–60 counts. Adjust the comparison directions if your circuit responds in the opposite direction.

### Hysteresis Test

Turn-on threshold: \_\_\_\_\_ Turn-off threshold: \_\_\_\_\_

How does the two-threshold behavior differ from the single-threshold behavior?

---

---

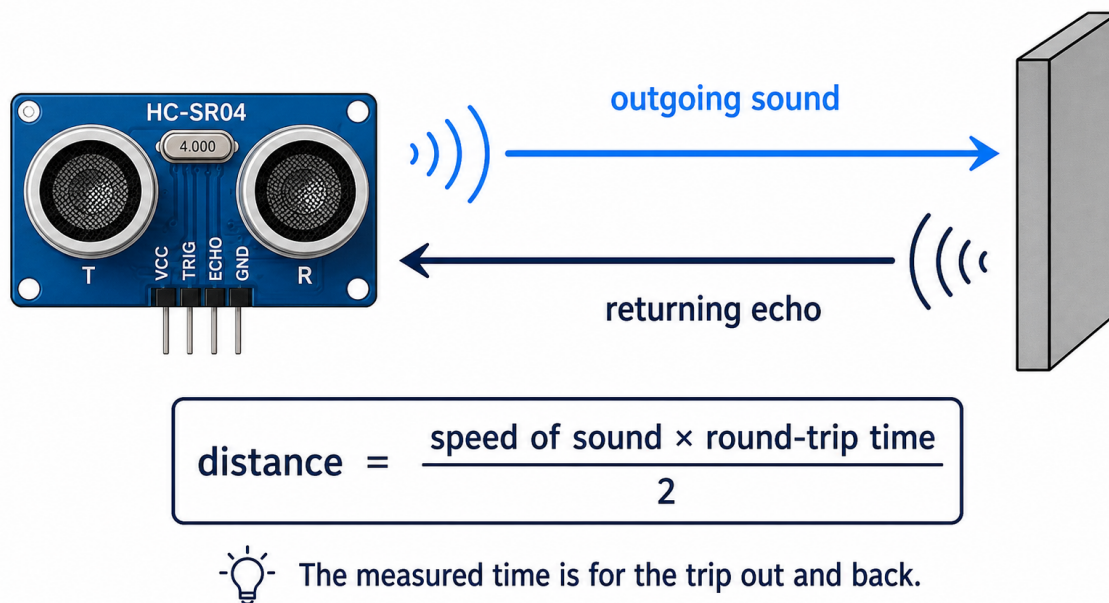
---

## Lab 3.2

### Ultrasonic Distance and Reliable Decisions

Time of Flight, Validation, Averaging, and Warning Zones

#### 7. Part E: Wire the HC-SR04 Sensor



| HC-SR04 pin | Purpose                    | Arduino connection |
|-------------|----------------------------|--------------------|
| VCC         | Sensor power               | 5V                 |
| GND         | Electrical reference       | GND                |
| TRIG        | Trigger pulse from Arduino | digital pin 7      |
| ECHO        | Return pulse to Arduino    | digital pin 6      |

#### Wiring Steps

1. Disconnect USB power.
2. Identify the pin labels printed on the sensor module.
3. Connect VCC to 5V and GND to GND.
4. Connect TRIG to digital pin 7.



5. Connect ECHO to digital pin 6.
6. Place a large flat object approximately 30 cm in front of the sensor.
7. Inspect the wiring before reconnecting power.

## 8. Part F: Measure Distance

```
1  const int trigPin = 7;
2  const int echoPin = 6;
3
4  void setup() {
5      pinMode(trigPin, OUTPUT);
6      pinMode(echoPin, INPUT);
7      Serial.begin(9600);
8  }
9
10 void loop() {
11     digitalWrite(trigPin, LOW);
12     delayMicroseconds(2);
13
14     digitalWrite(trigPin, HIGH);
15     delayMicroseconds(10);
16     digitalWrite(trigPin, LOW);
17
18     unsigned long duration =
19         pulseIn(echoPin, HIGH, 30000);
20
21     if (duration == 0) {
22         Serial.println("No valid echo");
23     } else {
24         float distanceCm =
25             duration * 0.0343 / 2.0;
26         Serial.println(distanceCm);
27     }
28
29     delay(100);
30 }
```

## Instructions

---

1. Upload the program and open the Serial Monitor.
2. Place a large flat target at approximately 10 cm, 25 cm, and 50 cm.
3. Use a ruler to measure the actual distance from the sensor face to the target.
4. Record five Arduino measurements at each reference distance.

---

**Distance Measurements**

---

| Reference | 1 | 2 | 3 | 4 | 5 | Average |
|-----------|---|---|---|---|---|---------|
| 10 cm     |   |   |   |   |   |         |
| 25 cm     |   |   |   |   |   |         |
| 50 cm     |   |   |   |   |   |         |

---

For one test distance, calculate the error:

$$\text{error} = \text{measured average} - \text{reference distance.}$$

Reference distance: \_\_\_\_\_

Measured average: \_\_\_\_\_

error = \_\_\_\_\_

Why is the echo travel distance divided by two?

---

---

---

**Checkpoint 3: Distance Measurement**

---

Demonstrate that the reported distance changes as the target moves. Explain the trigger pulse, echo pulse, and divide-by-two calculation.

## 9. Part G: Test Unreliable Targets

Compare the following targets at approximately the same distance:

| Target                            | Expected measurement behavior            |
|-----------------------------------|------------------------------------------|
| Large flat book facing the sensor | usually stable and reliable              |
| Narrow object or pencil           | may be missed or inconsistent            |
| Soft towel or fabric              | may absorb or scatter sound              |
| Book held at an angle             | may reflect sound away from the receiver |

### Target Comparison

| Target        | Typical reading | Stable or unstable? |
|---------------|-----------------|---------------------|
| Flat book     |                 |                     |
| Narrow object |                 |                     |
| Soft object   |                 |                     |
| Angled book   |                 |                     |

Which target produced the most reliable reading, and why?

---



---



---

## 10. Part H: Average Valid Readings

Use a function that collects five valid measurements and returns their average.

```

1 float averageDistance(int samples) {
2     float sum = 0.0;
3     int validCount = 0;
4
5     for (int i = 0; i < samples; i++) {
6         float d = readDistanceCm();
7         if (d >= 2.0 && d <= 400.0) {
8             sum += d;
9             validCount++;
10        }
11        delay(30);
12    }
13
14    if (validCount == 0) return -1.0;
15    return sum / validCount;
16 }
```

### Averaging Results

Hold a flat target at approximately 30 cm.

| Method              | Smallest reading | Largest reading |
|---------------------|------------------|-----------------|
| Single measurements |                  |                 |
| Five-sample average |                  |                 |

Which method varied less?

---



---

What is one disadvantage of averaging more samples?

---



---

## 11. Part I: Build a Distance Warning System

Keep the photoresistor circuit disconnected. Add an LED to PWM pin 9 through a  $330\,\Omega$  resistor.

```

1  const int ledPin = 9;
2
3  void loop() {
4      float distanceCm = averageDistance(5);
5
6      if (distanceCm < 0) {
7          analogWrite(ledPin, 0);
8      }
9      else if (distanceCm < 15) {
10         analogWrite(ledPin, 255);
11     }
12     else if (distanceCm < 35) {
13         analogWrite(ledPin, 100);
14     }
15     else {
16         analogWrite(ledPin, 0);
17     }
18 }

```

### Warning-Zone Results

| Distance region      | PWM command | LED appearance |
|----------------------|-------------|----------------|
| Below 15 cm          | 255         |                |
| 15 cm to below 35 cm | 100         |                |
| 35 cm and farther    | 0           |                |
| Invalid reading      | 0           |                |

### Checkpoint 4: Distance Warning System

Demonstrate the danger, warning, and safe regions. Explain why an invalid reading should not be treated as an object at zero distance.

## 12. Part J: Design Challenge

### Design Challenge

---

Choose one challenge:

1. **Adjustable warning boundary:** Use the photoresistor reading to select between two distance-warning thresholds.
2. **Blink-rate warning:** Make the LED blink faster as an object approaches the ultrasonic sensor.
3. **Stable alarm:** Add hysteresis so the danger alarm turns on below 18 cm and turns off above 22 cm.
4. **Repeated confirmation:** Require three consecutive close readings before activating the danger state.

### Design Record

---

Which challenge did you choose?

---

Describe the program changes.

---

---

Record one test condition and the resulting behavior.

---

---

What problem did you encounter, and how did you troubleshoot it?

---

---

---

## 13. Final Reflection

Explain how Lab 3 demonstrates the sense-decide-act model. Include one example of measurement validation or filtering that made the decision more reliable.

---

## 14. Submission Checklist

- ☐ Completed Lab 3 instruction sheet
- ☐ Photoresistor calibration measurements and selected threshold
- ☐ Ultrasonic reference-distance measurements
- ☐ Averaging and target-comparison results
- ☐ Clear photo of the final circuit
- ☐ Final Arduino code
- ☐ Completed design challenge and reflection